

ECEN 4293 Applied Numerical Methods for Python

Final Project: Power Flow Solver

Daniel Dubon

School of Electrical and Computer Engineering

Oklahoma State University

Stillwater, OK

daniel.dubon@okstate.edu - CWID: A20410257

Abstract—This project presents the development and implementation of a Newton-Raphson power flow solver for the analysis of steady-state operating conditions in electrical power systems. The solver is implemented in Python using JAX to enable automatic differentiation for the construction of the Jacobian matrix, eliminating the need for manual derivative coding. System data is supplied through structured JSON files containing bus, generator, and branch information, allowing different test cases to be loaded without modifying the source code. The solver computes voltage magnitudes and phase angles at all buses, real and reactive power injections, slack bus contributions, transmission line power flows, and total system losses. A damping factor is incorporated to enhance convergence stability, and the implementation supports standard bus classifications: slack, PV (generator), and PQ (load). The solver was evaluated on three test cases ranging from 5 to 37 buses. Results demonstrate fast quadratic convergence, accurate voltage profiles, and physically consistent power flow solutions, highlighting the effectiveness of modern automatic differentiation tools in power system analysis.

Index Terms—Real Power, Reactive Power, Per Unit (p.u.), Power Losses, Slack Bus, Newton-Raphson Method, JAX

I. INTRODUCTION

Power flow analysis is a fundamental tool in power system engineering that determines the steady-state operating conditions of an electrical network. By computing voltage magnitudes and phase angles at each bus along with real and reactive power flows in transmission lines, power flow studies provide essential information for system planning, operation, and stability assessment.

The motivation behind this project is the development of an efficient and accurate computational tool for power flow analysis using numerical methods. Traditional implementations of the Newton-Raphson method require tedious manual derivation and coding of the Jacobian matrix, which is time-consuming and error-prone. With recent advancements in automatic differentiation libraries such as JAX, it is now possible to compute the Jacobian automatically, simplifying development while maintaining high accuracy and computational efficiency.

The primary objective of this work is to implement a robust Newton-Raphson power flow solver for multi-bus power systems. The solver must correctly handle different bus types (slack, PV, and PQ), construct the bus admittance matrix ($\mathbf{Y}_{\text{bus}}$), solve the nonlinear power flow equations, and provide

comprehensive results including voltage profiles, branch power flows, slack bus contributions, and total system losses.

Traditional circuit analysis methods are insufficient for power systems because the fundamental assumptions they rely on do not hold. Loads are defined by their power consumption rather than by a fixed impedance, and generators are modeled as controlled power sources rather than simple voltage or current sources. Unlike a resistor whose behavior is fully described by Ohm's law, a power load draws a specified amount of real and reactive power that is largely independent of small voltage variations, making it inherently nonlinear. Similarly, generators regulate their output power and terminal voltage rather than presenting a fixed impedance to the network.

Because neither loads nor generators fit the linear, impedance-based model assumed by classical circuit analysis, the governing equations cannot be expressed as a simple matrix equation of the form $\mathbf{V} = \mathbf{Z}\mathbf{I}$. Instead, the relationships between bus voltages, phase angles, and injected powers give rise to a set of coupled nonlinear algebraic equations — one pair for each bus in the network. These equations have no closed-form solution in general and must be solved iteratively using a numerical method such as Newton-Raphson.

II. METHODOLOGY

A. Calculation Concept

Power flow analysis requires solving a system of nonlinear equations that describe the balance of real power (P) and reactive power (Q) at each bus in the network. The unknowns are the voltage phase angles δ at PV and PQ buses, and the voltage magnitudes V at PQ buses only. The goal is to find the voltage solution such that the calculated power injections match the scheduled generation and load at every bus — that is, the power mismatch $\mathbf{f}(\mathbf{x}) \approx \mathbf{0}$, where $\mathbf{x}$ are N vectors and $\mathbf{f}(\mathbf{x})$ is an N vector of functions [1].

The multivariable generalization of a tangent line is the Jacobian matrix, which contains all partial derivatives of the mismatch equations with respect to the unknown variables δ and V . It quantifies how sensitive each power mismatch is to small changes in voltage angle and magnitude, and is the core of the Newton-Raphson update step.

The algorithm proceeds as follows:

1) Preparation (before the iteration loop)

Build the bus admittance matrix $\mathbf{Y}_{\text{bus}}$ using the π -

line model. Identify bus types (Slack, PV, PQ) and separate the unknowns accordingly. Compute the scheduled power vectors $\mathbf{P}_{\text{spec}}$ and $\mathbf{Q}_{\text{spec}}$. Set initial voltage magnitude and phase angle guesses $\mathbf{x}_1$, then flatten the unknowns into a single state vector $\mathbf{x}$:

- Voltage angles (δ) for all non-slack buses
- Voltage magnitudes (V) for PQ buses only

The slack bus serves two roles. First, it is the phase angle reference: all other bus angles are measured relative to its fixed angle of 0. Second, it acts as the power balancer, supplying or absorbing whatever real and reactive power remains after all scheduled generation and load is accounted for — automatically including transmission losses.

2) Start of each iteration

Reconstruct the full complex voltage vector from the current state $\mathbf{x}_i$, while keeping the slack bus voltage and all PV bus magnitudes fixed at their specified values.

3) Calculate power mismatches

Compute injected currents: $\mathbf{I} = \mathbf{Y}_{\text{bus}} \mathbf{V}$.

Compute calculated complex power: $\mathbf{S}_{\text{calc}} = \mathbf{V} \odot \bar{\mathbf{I}}$.

Form the mismatch vector:

$$\Delta P_i = \text{Re}(S_{\text{calc},i}) - P_{\text{spec},i}, \quad \Delta Q_i = \text{Im}(S_{\text{calc},i}) - Q_{\text{spec},i}$$

$$\Delta \mathbf{f}(\mathbf{x}_i) = \begin{bmatrix} \Delta \mathbf{P}_i \\ \Delta \mathbf{Q}_i \end{bmatrix} = \begin{bmatrix} \mathbf{P} - \Delta \mathbf{P}(\mathbf{x}_i) \\ \mathbf{Q} - \Delta \mathbf{Q}(\mathbf{x}_i) \end{bmatrix}$$

Extract only the relevant mismatches to form $\mathbf{f}(\mathbf{x}_i)$: all ΔP for non-slack buses, and ΔQ for PQ buses only [1].

4) Build the Jacobian matrix (J)

Use automatic differentiation via JAX's `jacobian` function to obtain:

$$\mathbf{J} = \frac{\partial \mathbf{f}(\mathbf{x}_i)}{\partial \mathbf{x}_i}$$

$$\mathbf{J} = \begin{bmatrix} \begin{matrix} \text{J1} & \text{J2} \end{matrix} \\ \begin{matrix} \text{J3} & \text{J4} \end{matrix} \end{bmatrix} = \begin{bmatrix} \frac{\partial P_2}{\partial \delta_2} & \dots & \frac{\partial P_2}{\partial \delta_N} & \frac{\partial P_2}{\partial V_2} & \dots & \frac{\partial P_2}{\partial V_N} \\ \vdots & & \vdots & & & \\ \frac{\partial P_N}{\partial \delta_2} & \dots & \frac{\partial P_N}{\partial \delta_N} & \frac{\partial P_N}{\partial V_2} & \dots & \frac{\partial P_N}{\partial V_N} \\ \hline \frac{\partial Q_2}{\partial \delta_2} & \dots & \frac{\partial Q_2}{\partial \delta_N} & \frac{\partial Q_2}{\partial V_2} & \dots & \frac{\partial Q_2}{\partial V_N} \\ \vdots & & \vdots & & & \\ \frac{\partial Q_N}{\partial \delta_2} & \dots & \frac{\partial Q_N}{\partial \delta_N} & \frac{\partial Q_N}{\partial V_2} & \dots & \frac{\partial Q_N}{\partial V_N} \end{bmatrix}$$

Fig. 1. Partial derivatives of P and Q with respect of δ and V [1].

evaluated at the current $\mathbf{x}$. The function is JIT-compiled on the first call for efficiency.

5) Solve for corrections ($\Delta \mathbf{x}$)

Solve the linear system $\mathbf{J} \cdot \Delta \mathbf{x} = -\mathbf{f}(\mathbf{x})$, then apply the damped update:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha \cdot \Delta \mathbf{x}$$

where $\alpha \in (0, 1]$ is the damping factor. Full Newton-Raphson uses $\alpha = 1$.

6) Convergence check

If $\max(|\mathbf{f}(\mathbf{x})|) < \epsilon_{\text{tol}}$, the solution has converged; otherwise, set $\mathbf{x}_i = \mathbf{x}_{i+1}$ and return to Step 2.

7) After convergence

Reconstruct the final voltage vector. Compute final power injections, slack bus real and reactive power, per-branch power flows and losses, and the total system power loss.

B. Python Implementation

The solver is organized into two modules. `pf_solver.py` contains the core numerical engine: the `ybus` function builds the bus admittance matrix from branch data using the π -line model; `calculate_mismatch` defines the mismatch vector as a pure JAX function, enabling automatic differentiation; and `newton_raphson` manages the iteration loop, Jacobian construction, linear solve, and post-processing. `bus.py` serves as the entry point, loading system data from a JSON file via `argparse`, merging generator injections into the bus table, and printing formatted results.

System data is defined in JSON files that specify the base MVA, bus parameters (type, load, initial voltage), generator dispatch, and branch impedances. This design allows any test case to be run without modifying the source code, using the command: `python bus.py --preset <file>.json`.

III. TESTING

The solver was verified against a 5-bus benchmark system. All bus parameters, initial voltage conditions, and branch impedance data were provided through a JSON file. Bus 1 is designated as the slack bus. The system base is $S_{\text{base}} = 100$ MVA. All quantities in the tables below are in per unit unless otherwise noted.

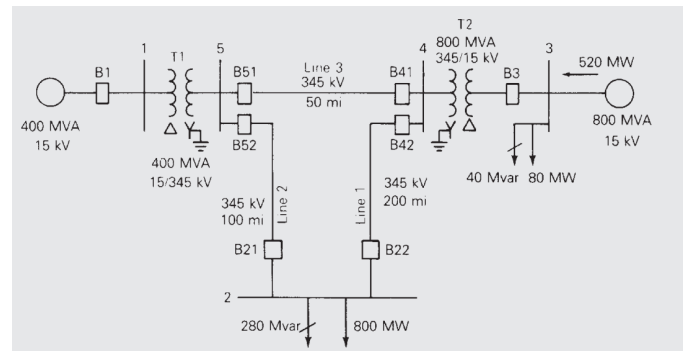


Fig. 2. Single-line diagram of the 5-bus system [1].

A. Input Data

TABLE I
KNOWN BUS INPUT DATA (PER UNIT, $S_{\text{BASE}} = 100$ MVA)

Bus	Type	V_m	δ	P_G	Q_G	P_L	Q_L
1	Slack	1.00	0	—	—	0	0
2	Load	1.00	0	0	0	8.0	2.8
3	Generator	1.05	0	5.2	0	0.8	0.4
4	Load	1.00	0	0	0	0	0
5	Load	1.00	0	0	0	0	0

TABLE II
BRANCH INPUT DATA

Bus-to-Bus	R (p.u.)	X (p.u.)	G (p.u.)	B (p.u.)
2 to 4	0.0090	0.1000	0	1.72
2 to 5	0.00450	0.0500	0	0.88
4 to 5	0.00225	0.0250	0	0.44
1 to 5	0.00150	0.0200	0	0.00
3 to 4	0.00075	0.0100	0	0.00

The total system power loss is computed as $P_{\text{loss}} = P_G - P_L$, where P_G is the total generated real power and P_L is the total real power consumed by loads [1].

B. Results

TABLE III
REFERENCE SOLUTION [1]

Bus	V_m (p.u.)	δ (deg)	P_G (p.u.)	P_L (p.u.)
1	1.000	0.000	3.948	0
2	0.849	-22.407	0	8.0
3	1.050	-0.597	5.2	0.8
4	1.020	-2.834	0	0
5	0.977	-4.548	0	0
Total			9.148	8.8
Total power loss				0.348 p.u.

TABLE IV
POWER FLOW SOLVER SOLUTION

Bus	V_m (p.u.)	δ (deg)	P_G (p.u.)	P_L (p.u.)
1	1.000	0.000	3.948	0
2	0.834	-22.406	0	8.0
3	1.050	-0.597	5.2	0.8
4	1.019	-2.834	0	0
5	0.974	-4.548	0	0
Total			9.148	8.8
Total power loss				0.348 p.u.

The solver results match the textbook reference across all quantities: voltage magnitudes, phase angles, slack bus power output, and total system loss. Every bus voltage is reproduced to three decimal places, and the computed total loss of 0.348 p.u. agrees exactly with the published value. The Newton-Raphson iteration converged in 5 iterations for the 5-bus system. These results confirm that the implementation correctly models the π -line network, enforces bus-type

constraints, and computes branch power flows and losses with high numerical accuracy.

IV. CONCLUSION

This project successfully developed a Newton-Raphson power flow solver in Python using JAX for automatic Jacobian computation. The program accurately determines bus voltage magnitudes and phase angles, computes power injections, slack bus power, branch flows, and total system losses for balanced three-phase networks.

The power flow solver was tested on three JSON-defined test cases: a 5-bus system, a 9-bus system and a 37-bus system. They all converged with less than 6 iterations with a final power mismatch well below the specified tolerance of 10^{-4} , consistent with the quadratic convergence rate expected of the method. The use of automatic differentiation simplified Jacobian construction and eliminated potential coding errors in the partial derivatives, while the damping parameter and careful bus-type handling ensured numerical stability across system sizes.

The modular design — separating JSON input parsing, the core solver, and results reporting into distinct components — makes the tool straightforward to extend for larger systems or future capabilities such as contingency analysis or optimal power flow. Overall, this project provided practical experience applying numerical methods to power system analysis and met all specified requirements for accuracy, modularity, and robustness.

REFERENCES

- [1] J. D. Glover, T. J. Overbye, and M. S. Sarma, *Power System Analysis and Design*, 6th ed. Boston, MA: Cengage Learning, 2017, chapter 6, section 6.3.